Hertie School

Natural Language Processing

Session 7: Neural Models I
Topic models, closer · neural networks · CNNs

GRAD-E1282 · Fall Semester 2026
Luis Fernando Ramírez Ruiz · l.ruiz@hertie-school.org

Announcements

Research Note one-pager:

- The week after coming back from break (session 8).

Quiz 3: Week on session 9:

- Cover session 6, 7, and 8

Today: finish topic models, then let the network learn

1 PART 1 · ≈ 30 MIN

Topic models, closer

What LDA learns and maximises; UMAP, HDBSCAN and c-TF-IDF intuitively; LDA vs BERTopic.

2 PART 2

Neural networks, reviewed

From logistic regression to hidden layers, and how backpropagation trains them.

3 PART 3

PyTorch and CNNs

What PyTorch solves; how a CNN filter works, from images to text. Session 8 goes deeper.

WHERE SESSION 6 STOPPED

- LDA's generative story: θ for documents, β for topics.
- BERTopic as a framework that turns topic modelling into clustering.
- UMAP and HDBSCAN, in outline.

WHAT WE ADD TODAY

- What LDA learns (θ and β), what it maximises, and how it is fitted.
- What UMAP, HDBSCAN and c-TF-IDF each do, intuitively.
- When to choose which model.

By the end: choose between LDA and BERTopic and defend the settings; explain how a hidden layer learns a new representation; explain what a CNN filter does.

LDA: what it learns and what it maximises

Every document mixes a few topics; every topic mixes a few words. LDA looks for the topics and the mixes that best explain the words we actually see.

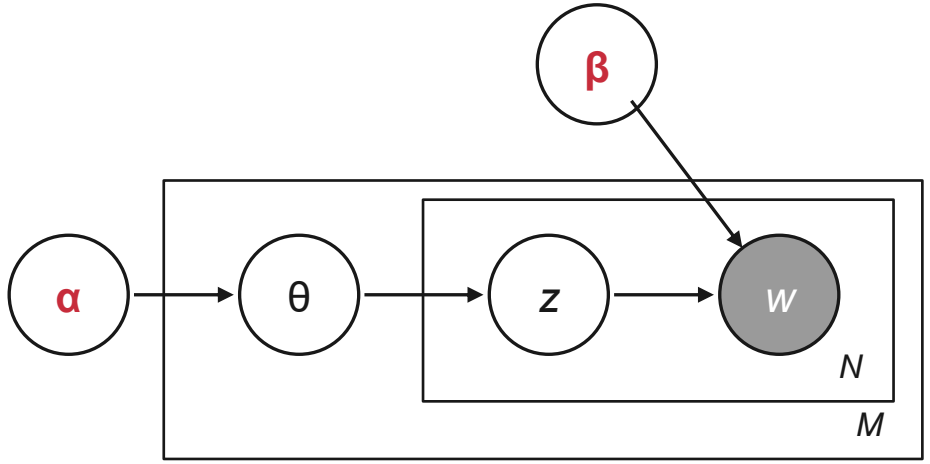


Figure 1 of Blei, Ng & Jordan (2003), redrawn. Shaded: observed.

α Corpus level, outside the plates. How concentrated each document's mix is: the Dirichlet prior on θ.

β Corpus level, outside the plates. A k × V table: β_{ij} = p(word j | topic i). Not drawn from anything: estimated from the corpus.

θ One per document (plate M): its topic mix, θ ~ Dirichlet(α).

z, w One per word (plate N): pick a topic z from θ, then the word w from row z of β.

what LDA maximises

$$p(D \mid \alpha, \beta) = \prod_d \int p(\theta_d \mid \alpha) \left(\prod_n \sum_z p(z_{dn} \mid \theta_d)\, p(w_{dn} \mid z_{dn}, \beta) \right) d\theta_d$$

α and β are fixed for the whole corpus; θ and z are hidden, per document and per word. LDA chooses α and β to make the observed corpus as probable as possible. The integral has no exact solution, so it is fitted approximately: next slide.

Blei's smoothed variant (Figure 7) gives β its own Dirichlet prior η; libraries call it eta or topic_word_prior.

How LDA is fitted: Gibbs sampling

INPUT: the corpus D, k topics, α

1. Give every word w_{dn} a random topic z_{dn}; compute θ and β from these topics.
2. Repeat for many sweeps. For every word w_{dn}:
   - **a** score every topic i with the formula on the right
   - **b** draw a new topic z_{dn} in proportion to the scores
3. After each sweep, re-estimate θ and β from the new topics.
4. Stop when the topics settle.

STEP 2a, THE ONLY FORMULA

$$p(z_{dn} = i \mid \theta_d, \beta) \propto \theta_{di} \cdot \beta_{ij}$$

where $j = w_{dn}$, the word itself

θ_{di}
how much document d uses topic i

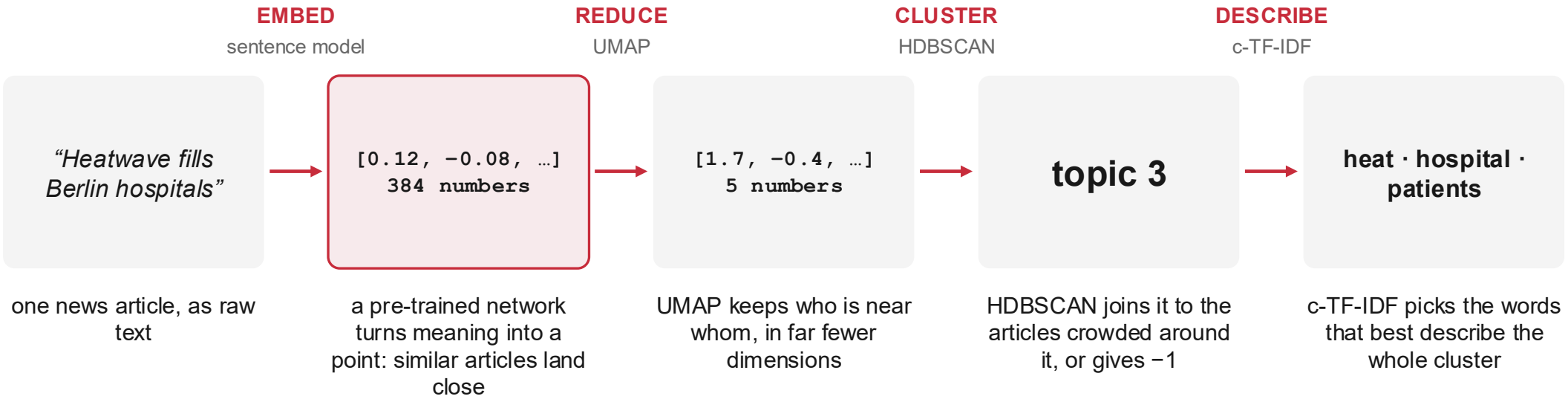
β_{ij}
how much topic i uses word j, across the corpus

Session 6's example: "heat" in a mostly-climate article still turns health 65% of the time, because health uses "heat" far more.

Only counting and resampling. Each sweep makes documents use fewer topics and topics use fewer words; when the topics settle, θ and β are the answer.

Same symbols as slide 4 (Blei, Ng & Jordan, 2003): w_{dn} is word n of document d, z_{dn} its topic.

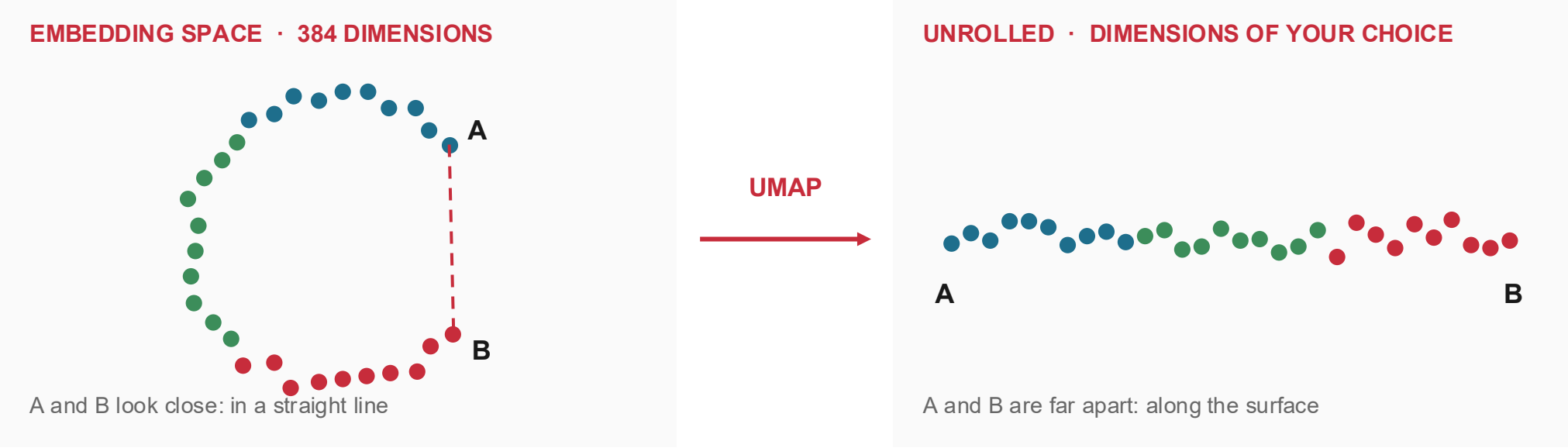
BERTopic: follow one article through the four steps



Only the embedding step is a neural network, and it was trained on other text: BERTopic borrows it. The next three slides open the other steps.

Each step is a separate model you can swap: a multilingual embedding model, different UMAP or HDBSCAN settings, different stop words for the description.

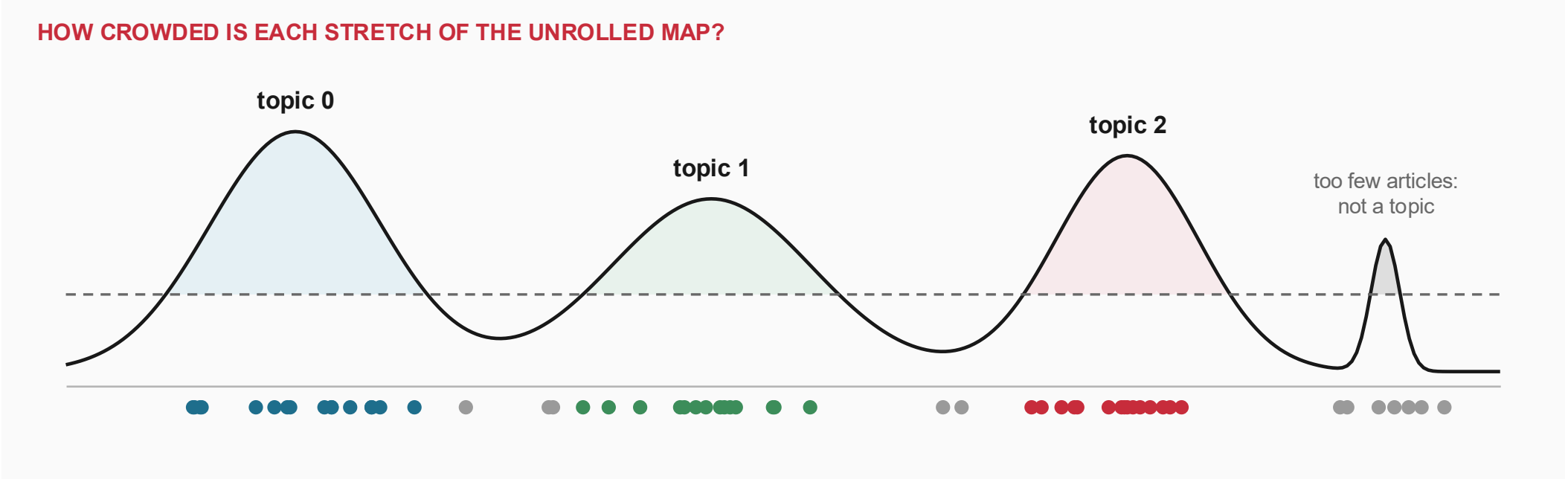
UMAP: unroll the surface the articles live on



UMAP is a **non-linear dimensionality reduction**. From each article's nearest neighbours it approximates the curved surface the articles lie on, then unrolls it into as many dimensions as you choose (n_components): 2 to plot it, 5 by default in BERTopic to cluster it. A linear method such as LSA or PCA would flatten the ring instead, and put A on top of B.

UMAP keeps who is near whom along the surface. That is all the clustering step needs.

HDBSCAN: on that map, find the crowds



HDBSCAN asks how crowded each region of the UMAP map is, then **raises the bar** (dashed line) step by step. Crowds that survive longest become topics; articles in sparse regions (grey) stay −1.

UMAP draws the map; HDBSCAN finds the crowds on it. No K to choose, and no article forced into a topic.

c-TF-IDF: which words best describe each cluster?

1 · JOIN

Glue each cluster's articles into one long text: three clusters, three texts.

2 · SCORE EVERY WORD

$W(t, c) = tf(t, c) \cdot \log(1 + A / tf(t))$: frequent in this cluster, rare across the others.

3 · READ THE TOP WORDS

Sort each cluster's scores. The highest-scoring words become the topic's label.

TOY EXAMPLE: CLUSTER c_1 = "HEAT AND HEALTH", 3 CLUSTERS × 100 WORDS, SO A = 100

term	tf in c_1	tf in all	log(1 + A/tf)	c-TF-IDF
heat	8	12	2.23	**17.9**
hospital	6	7	2.73	**16.4**
price	1	11	2.31	**2.3**
the	30	90	0.75	**22.4**

A = average words per cluster · tf(t) = frequency of t across all clusters

WHAT YOU GET: WORDS PER TOPIC

topic 0	`hospital · heat · patients`
topic 1	`drought · harvest · rain`
topic 2	`price · energy · gas`

It scores words, never articles. To see which articles stand for a topic, use get_representative_docs().

"the" ranks first in the raw table: common words are damped, never erased. Remove stop words in the CountVectorizer only; the embeddings still see the full text.

LDA vs BERTopic, head to head

	LDA	BERTOPIC
A TOPIC IS	a probability distribution over the vocabulary	a dense region of documents, described afterwards by words
INPUT	raw word counts; remove stop words first	embeddings of the full text; stop words matter only for the word lists
HOW MANY TOPICS	K, set before fitting	emerges from min_cluster_size; merge with nr_topics
MIXTURES	every document is a mix θ	one topic per document; approximate_distribution() estimates mixes afterwards
OUTLIERS	none: every word gets a topic	−1 for documents that fit nowhere
SHORT TEXTS	weak: too few words to estimate θ	strong: meaning comes from pre-training
SEVERAL LANGUAGES	one vocabulary per language; topics do not line up	a multilingual embedding model puts all languages in one space
COST	CPU, minutes	embedding is slow without a GPU: cache it

Choose by your documents and your question. Long texts that mix themes: LDA. Short, multilingual or noisy texts: BERTopic. Either way, report every setting you chose.

Discuss: LDA or BERTopic for your research note?

1 Describe your corpus in one line: how many documents, how long, which languages. Which model fits it better, and which row of the last slide decided it?

2 A BERTopic run leaves 40% of your articles as −1. Is that a problem for your research question? Would you change min_cluster_size, the embedding model, or nothing?

3 LDA says an article is 60% climate and 40% health; BERTopic puts it in a single health topic. Which description is closer to how you would code it by hand?

3 min in pairs → 3 min discussion.

Evaluating topics without labels

No labels means no confusion matrix and no F1 (Session 5). Four questions instead, for either model:

1 COHERENCE

Do a topic's top words appear together in real documents?

NPMI, from −1 to 1

2 DIVERSITY

Do topics use different words, or repeat each other?

unique top words ÷ all top words

3 STABILITY

Do the same topics come back with another seed or a 90% subsample?

overlap across runs

4 HUMAN READING

Can a colleague name the topic and spot an intruder word?

`heat · hospital · patients · invoice`

LDA ONLY · PERPLEXITY

How well slide 4's p(D | α, β) predicts held-out documents. Useful for fitting, but it can disagree with what people find readable.

BERTOPIC ONLY · SHARE OF −1

How many articles fit no topic. Report it next to the topics: a clean model that ignores half the corpus is not a clean model.

Proxies, not ground truth. Report coherence and diversity together, on the same reference corpus for both models, check stability, and read the documents.

From LDA to BERTopic to today: who builds the representation?

One question separates the three: **who builds the representation, and what does it answer to?**

LDA · SESSION 6

counted →

representation: word counts

per document: **K topic proportions, fitted to reproduce its words**

Built from your corpus alone. No label is involved at any point.

BERTOPIC · SESSION 6

borrowed →

representation: a sentence embedding

per document: **one vector from a network pre-trained on other text**

We clustered the vectors but never changed the network. It stayed a black box.

NEURAL NETWORK · TODAY

learned

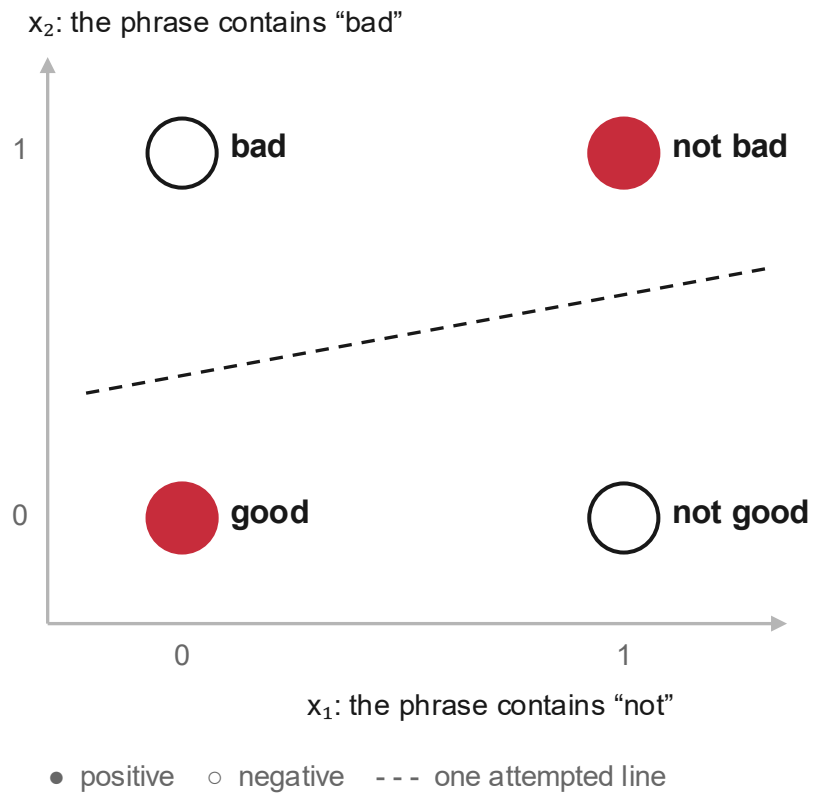
representation: hidden layers

per document: **a vector computed by layers trained on your labels**

The representation and the classifier are fitted together, end to end.

The difference is what the middle layer answers to. LDA's topics answer to the word counts; BERTopic's embedding answers to a pre-training task on someone else's text; a hidden layer answers to your label: it keeps what helps predict y and drops the rest. Today we open the box BERTopic borrowed.

Why linear is not enough: the problem of “not bad”



WHY NO LINE WORKS

The two positives sit on a diagonal, the two negatives on the other. Any straight line that keeps “good” and “not bad” together also catches a negative: the XOR pattern from the reading.

SESSION 5’S FIX

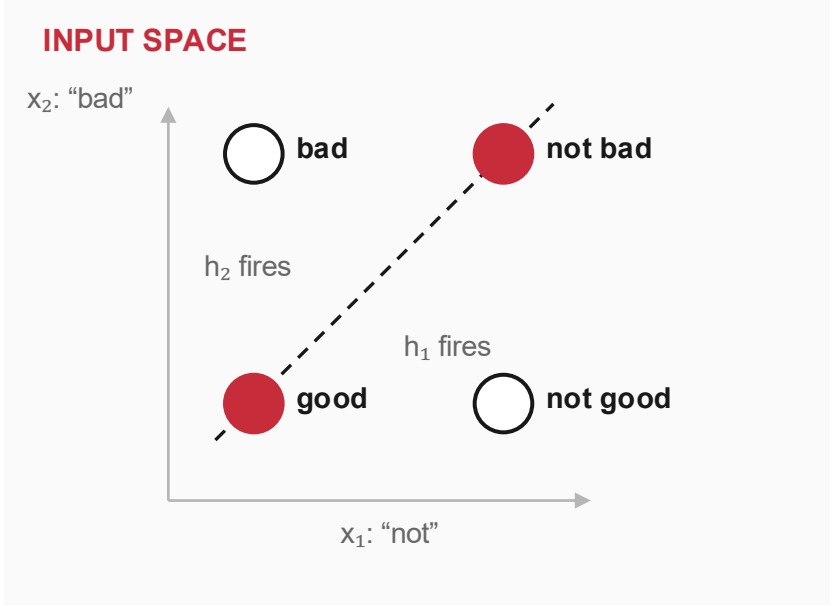
Engineer the interaction: add the bigram “not bad” as its own column. It works, if you foresee every pair that matters.

TODAY’S FIX

Let a hidden layer learn the interaction. In the reading, a model with no hidden layer stays near 70% validation accuracy on XOR; two hidden layers of four units reach 95%.

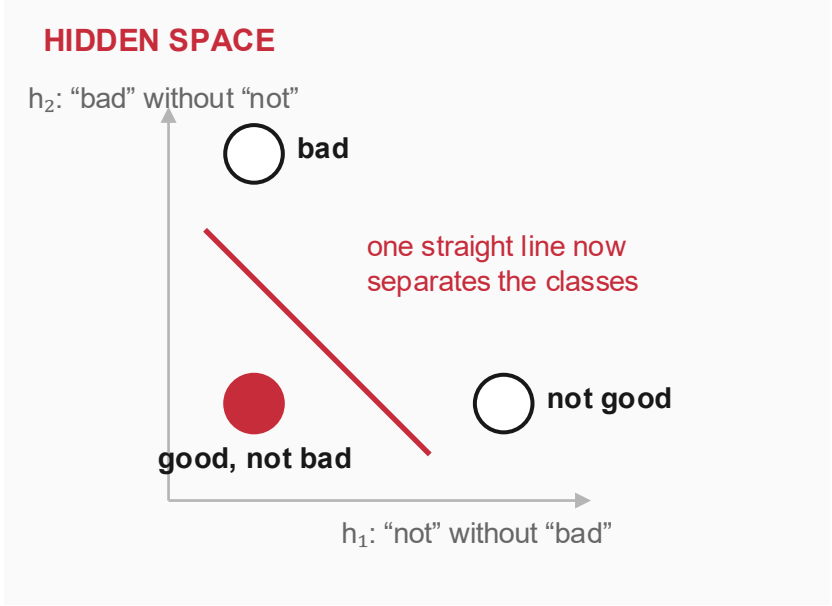
Session 5 ended on this limit: a linear model cannot discover that two features matter only together. A network with one hidden layer can.

A hidden layer moves the data into a new space



$$h = \mathrm{ReLU}(W_1 x + b_1)$$

$$h_1 = \mathrm{ReLU}(x_1 - x_2)$$
$$h_2 = \mathrm{ReLU}(x_2 - x_1)$$



Each neuron draws a hyperplane, like a logistic regression, but reports where the input falls instead of deciding the class: a new coordinate.

Each layer re-describes every input by those coordinates: a learned representation, trained only because it helps the prediction.

Deep networks repeat this many times. No single coordinate stays readable: that is the black box.

Logistic regression finds a line in the space you give it; a neural network first learns the space.

The universal approximation theorem: why f matters

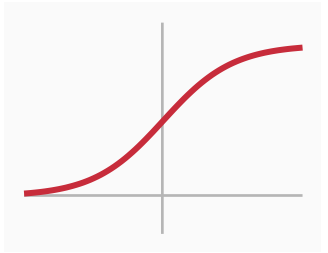
Universal approximation theorem (Cybenko, 1989; Hornik, Stinchcombe & White, 1989). A network with **one hidden layer**, a non-linear activation f and enough units can approximate any continuous function as closely as you like.

WITHOUT f, LAYERS COLLAPSE

$$W_2(W_1x + b_1) + b_2$$
$$= (W_2W_1)x + (W_2b_1 + b_2)$$
$$= W\,x + b$$

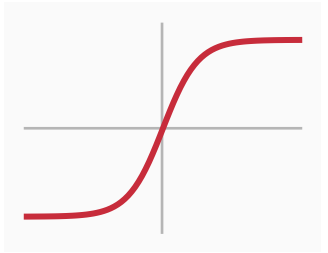
Two linear layers are one linear layer: still a single straight boundary.

THREE CHOICES OF f



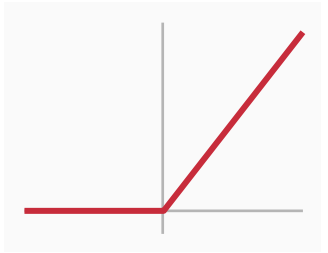
sigmoid

From 0 to 1; flat at both ends.



tanh

From −1 to 1; also flat at the ends.



ReLU

max(0, z): the default today.

WHAT IT DOES NOT PROMISE

That gradient descent will find those weights, or how many units it takes. In practice, deeper networks need far fewer units than one very wide layer.

Discuss: what is a hidden layer learning?

1 Take a sheet of paper and mark “good” and “not bad” in two opposite corners, “not good” and “bad” in the other two. Fold it once so that a single straight cut separates the two pairs. What did the fold do?

2 Hidden units are questions the network learns to ask about a text, like “is there a negation before a negative word?”. Write down two such questions that would help classify your own documents.

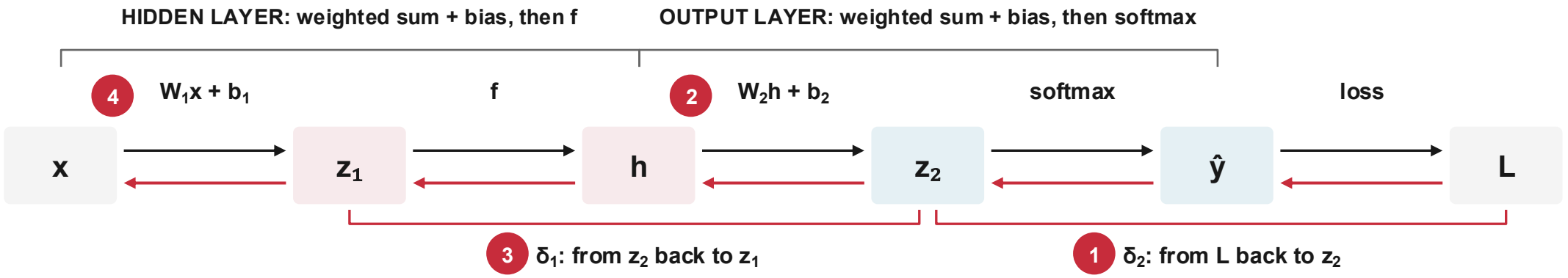
3 A network is right 2% more often than logistic regression but cannot tell you why. Would you use it to sort news for exploration? To remove posts as hate speech? Why the difference?

3 min in pairs → 3 min discussion.

Session Break

When we return: how networks learn (backpropagation), then PyTorch and a first look at convolutions.

Backpropagation: blame flows backwards



THE BACKWARD PASS, RIGHT TO LEFT (THE RED ARROWS)

1	**output error**	$\delta_2 = \partial L/\partial z_2 = \hat{y} - y$	lands at z_2: loss and softmax in one step
2	**output-layer gradients**	$\partial L/\partial W_2 = \delta_2 h^T$ $\quad \partial L/\partial b_2 = \delta_2$	for the weights on the arrow $h \rightarrow z_2$
3	**error at the hidden layer**	$\delta_1 = \partial L/\partial z_1 = (W_2^T \delta_2) \odot f'(z_1)$	back through W_2 to h, then through f to z_1
4	**hidden-layer gradients**	$\partial L/\partial W_1 = \delta_1 x^T$ $\quad \partial L/\partial b_1 = \delta_1$	for the weights on the arrow $x \rightarrow z_1$

Errors (1, 3) travel backwards from node to node; gradients (2, 4) are read off where an error meets the weights that produced it. Nothing goes back to x: it is data, not a weight.

Gradient descent: using the gradients to change the weights

THE LOSS AS A LANDSCAPE, FOR ONE WEIGHT



each step goes against the slope; steps shrink as it flattens

THE UPDATE RULE

$$w \leftarrow w - \eta \cdot \partial L/\partial w$$

for every weight and bias in every layer. Step against the slope: downhill. η, the learning rate, sets the step size.

THE TRAINING LOOP

1. Forward pass: compute ŷ for a batch of documents.
2. Loss: compare ŷ with the labels.
3. Backward pass: all gradients (previous slide).
4. Update every weight with the rule above.

Repeat for the next batch; one pass over all the data is an epoch.

Learning rate: too large and the steps jump across the valley; too small and training crawls. Unlike logistic regression, a network's landscape has many valleys, so the random start matters.

Backpropagation computes the gradients; gradient descent uses them. Session 5's logistic regression was trained by this same loop, with one layer.

PyTorch: what it solves, and why it runs most of deep learning

Torch (2002) → PyTorch, released by Meta AI (2016) → PyTorch Foundation (2022). Hugging Face and sentence-transformers, behind BERTopic, are built on it.

1 AUTOGRAD

You write only the forward pass. PyTorch records it and runs backpropagation for you: the backpropagation slide, automated.

2 GPU TENSORS

The same matrix code runs on a GPU's thousands of cores. Training that would take weeks takes hours.

3 BUILDING BLOCKS

Layers, losses and optimisers that snap together. Logistic regression is one linear layer; an MLP adds two lines.

SCIKIT-LEARN · SESSIONS 5–6

Ready-made models: .fit() hides the training loop.

PYTORCH · FROM TODAY

You define the model and write the loop: any architecture, at the price of more code.

CNNs in computer vision: a small filter slides over the image

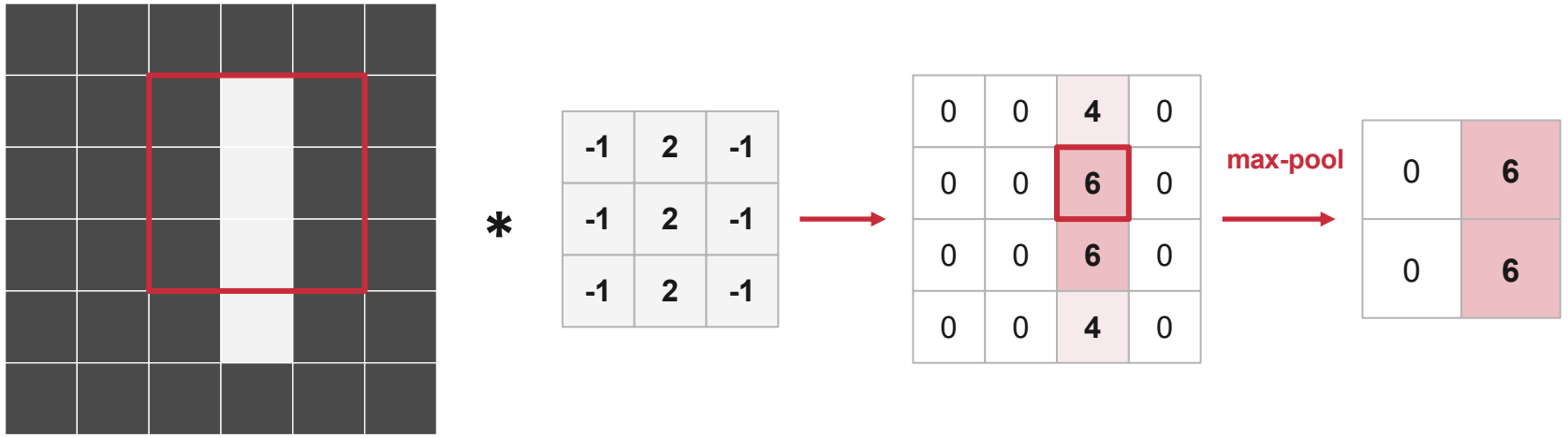


image · 36 pixels | kernel: finds vertical lines | feature map · 16 values | pooled · 4 values

The kernel is a small grid of weights that lights up where its pattern appears; the same kernel slides over every position. Each pass **summarises and loses detail**: 36 pixels → 16 → 4 numbers that say whether, and roughly where, a vertical line occurs.

Stack such layers: early kernels find edges, later ones shapes, then objects. The kernels are learned, not designed.

CNNs for text: how a 1D filter reads two words at a time

SENTENCE: 5 WORDS × 3 DIMS

	d_1	d_2	d_3
the	0.0	0.0	0.1
heat	0.1	0.4	0.0
was	0.0	0.0	0.1
not	**0.9**	**0.1**	**0.0**
bad	**0.0**	**0.9**	**0.0**

toy embeddings; real ones have 300 dims

FILTER: 2 WORDS × 3 DIMS

row 1: word 1 is a negation
row 2: word 2 is negative

1	**0**	**0**
0	**1**	**0**

bias b = −1

ONE POSITION: THE WINDOW “not bad”

1 · multiply cell by cell, then add everything

```
not: 0.9·1 + 0.1·0 + 0.0·0  =  0.9
bad: 0.0·0 + 0.9·1 + 0.0·0  =  0.9
```

2 · add the bias: 0.9 + 0.9 − 1 = 0.8

3 · ReLU(0.8) = 0.8: the pattern is there

The same as one neuron on the two vectors joined.

SLIDE TO EVERY POSITION, THEN POOL

the heat → 0 | heat was → 0 | was not → 0 | **not bad → 0.8** → **max-pool → 0.8**

Like a bigram, but learned and graded. Session 5’s “not bad” column was 0 or 1 for one exact pair. This filter scores any pair that looks like negation + negative word: “not terrible” scores 0.7, “was bad” scores 0.

A filter is one neuron for k-word windows, reused at every position. 100 filters are 100 learned phrase detectors, pooled into 100 numbers for the softmax.

What CNNs add that our earlier models could not

MODEL	WHERE CONTEXT COMES FROM	WHAT IS STILL MISSING
Logistic regression Session 5	None learned: each word adds its own weight to the logit, whatever its neighbours.	“not good” exists only if you added the bigram; “not terrible” is an unrelated feature.
Word2vec Session 4	From neighbouring words, in a shallow hidden layer, learned before the task.	One frozen vector per word; averaging a sentence throws the order away.
MLP today	Hidden layers learn combinations of features, for the task.	The input is still a bag of words: no order at all.
CNN today	Filters learn which short word sequences matter, for the task, read from embeddings, at every position.	Only k words at a time. Long-range links: Session 8.

What CNNs solve: n-grams the model learns instead of you listing them, that generalise across similar words (“not bad” ≈ “not terrible”), found anywhere in a sentence of any length.

Discuss: from gradients to filters

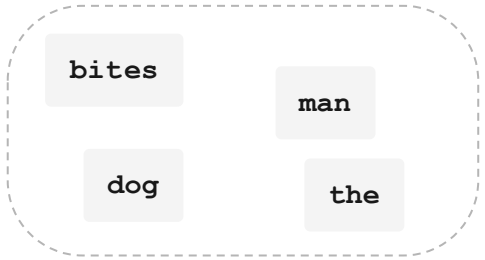
1 You walk down a foggy hill, feeling the slope with your feet. What happens with huge steps? With tiny ones? Sketch what the loss would look like over training in each case.

2 How wide must a filter be to read the negation in: “not bad”, “not a bad film”, “not, as many feared, a bad film”? What does that tell you about CNNs?

3 A filter fires on “not bad”. Will it also fire on “not terrible”? On “bad, not”? On the German “nicht schlecht”? Why or why not?

3 min in pairs → 3 min discussion.

What each network sees of word order



MLP · NONE

A bag of words: “dog bites man” and “man bites dog” look the same.

today

window of k = 2



CNN · LOCAL

Order inside a window: “not bad” differs from “bad, not”. Lost again after pooling.

today, then Session 8

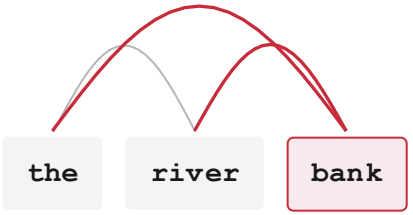
memory carried forward



RNN, LSTM · IN SEQUENCE

Reads one word at a time and carries a memory, so “not” can still matter words later.

Session 8



TRANSFORMER · EVERYWHERE

Every word reads every other: “bank” by the river is not “bank” at the vault.

Session 9

Each step to the right reads more of the order, and needs more data or pre-training. Today: none and local. Session 8: in sequence. Session 9: everywhere.

Where we are in the NLP pipeline



Data acquisition	Text extraction, normalization	Representation / features	Basic models	Neural models	Evaluation	Final evaluation + XAI	Orchestration
Not covered	Session 2	**Sessions 3–4**	Sessions 5–6	SESSIONS 7–9	Throughout	Session 10	Sessions 11-12

BERTopic PyTorch · next: CNNs, RNNs and LSTMs (Session 8)

Today: finish topic models and let the network learn the features. Next: networks that read text in order.